Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS



React Portals

Previous Next

React Portals provide a way to render HTML outside the parent component's DOM hierarchy.

This is particularly useful for components like modals, tooltips, and dialogs that need to break out of their container's layout.

What are React Portals?

SHARE

A Portal is a React method that is included in the `react-dom` package.

It is used to render HTML outside the parent component's DOM hierarchy.

Normally the returned HTML element is a child of the parent component, and returned like this:

```
  return (
    <div>
      Welcome
    </div>
  );
}
```

But by using the `createPortal` method, the HTML is not a child of the parent component, and is rendered outside the parent component's DOM hierarchy:

Example

With the `createPortal` method:

```
import { createPortal } from 'react-dom';

function myChild() {
  return createPortal(
    <div>
      Welcome
    </div>,
    document.body
  );
}
```

Syntax

```
import { createPortal } from 'react-dom';
```

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
HTML CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

The first argument (`children`) is any renderable React content, like elements, strings, or fragments.

The second argument (`domNode`) is a DOM element where the portal should be inserted instead.

Creating a Modal with Portal

React Portals are particularly useful for components like modals, tooltips, and dialogs that need to break out of their container's layout.

Here is an example of a modal component where the modal is rendered outside the parent component's DOM hierarchy:

Example

```
import { createRoot } from 'react-dom/client';
import { useState } from 'react';
import { createPortal } from 'react-dom';

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  return createPortal(
    <div style={{
      position: 'fixed',
      top: 0,
      left: 0,
      right: 0,
      bottom: 0,
      backgroundColor: 'rgba(0, 0, 0, 0.5)',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center'
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
          borderRadius: '8px'
        }}>
          {children}
          <button onClick={onClose}>Close</button>
        </div>
      </div>,
      document.body
    );
}

function MyApp() {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div>
      <h1>My App</h1>
      <button onClick={() => setIsOpen(true)}>
        Open Modal
      </button>

      <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
        <h2>Modal Content</h2>
        <p>This content is rendered outside the App component!</p>
      </Modal>
    </div>
  );
}

createRoot(document.getElementById('root')).render(
  <MyApp />
)
```

Run Example »

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

- `createPortal` from react-dom - for creating the portal
- `useState` from react - for managing the modal's open/close state

The `Modal` component uses `createPortal` to render its content directly into the `document.body` element.

REMOVE ADS

Why Use Portals

Portals are particularly useful for:

- Modals and dialogs
- Tooltips
- Floating menus
- Notifications

Any UI element that needs to "break out" of its container's layout, especially when the parent component has:

- `overflow: hidden`

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Event Bubbling in Portals

Even though a portal renders content in a different part of the DOM tree, events from the portal content still bubble up through the React component tree as if the portal wasn't there.

For example, if a button inside a portal is clicked, the event will still bubble up to the parent component, and the parent component's event handler will be triggered.

Example

Click the button to demonstrate that both the button and the parent div element will have its `onClick` event triggered:

```
import { createRoot } from 'react-dom/client';
import { useState } from 'react';
import { createPortal } from 'react-dom';

function PortalButton({ onClick, children }) {
  return createPortal(
    <button
      onClick={onClick}
      style={{
        position: 'fixed',
        bottom: '20px',
        right: '20px',
        padding: '10px',
        background: 'blue',
        color: 'white'
      }}>
      {children}
    </button>,
    document.body
  );
}
```

W3Schools app NEW iOS & Android
Sign In
Tutorials Exercises
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
  return (
    <div
      style={{
        padding: '20px',
        border: '2px solid black',
        margin: '20px'
      }}
      onClick={() => {
        setCount1(c => c + 1);
      }}>
      <h2>Div Clicked: {count1}</h2>
      <h2>Button Clicked: {count2}</h2>
      <p>The floating button is rendered outside this box using a portal,
          but its clicks still bubble up to this parent div!</p>
      <p>Try to click the div element as well, to see the count increase</p

      <PortalButton
        onClick={(e) => {
          // This runs first
          setCount2(c => c + 1);
        }}>
        Floating Button
      </PortalButton>
    </div>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

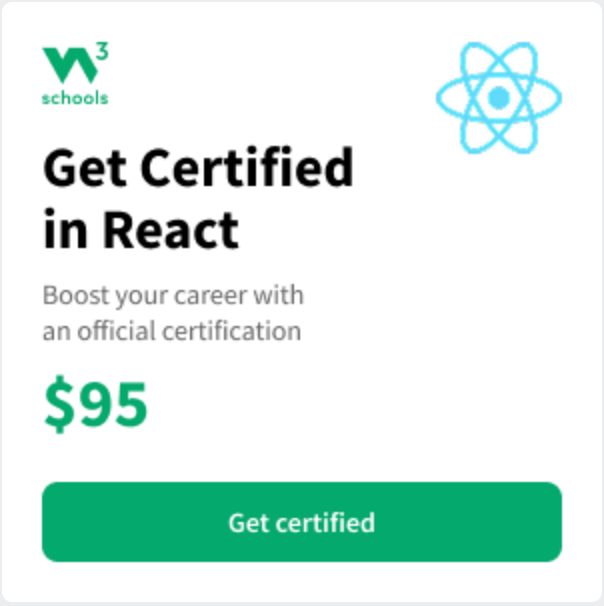
Run Example »

Example Explained

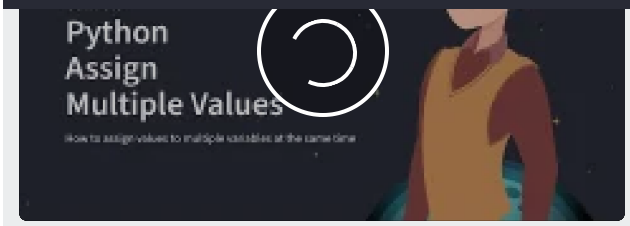
Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

- Even though the button exists outside the parent `<div>` in the DOM, clicking it will:
  1. First trigger the button's own `onClick` handler (incrementing the counter)
  2. Then trigger the parent div's `onClick` handler
- This demonstrates that event bubbling works through React's component hierarchy, not the DOM hierarchy

‹ Previous Sign in to track progress Next ›



REMOVE ADS



New customers get 50% off all unlimited data plans ($15/mo equiv. upfront, may slow...

SPONSORED BY MINTMOBILE.COM Learn More



FREE ACCOUNT

Track your progress

XP earned 0

NEW

Learn in Adventure Mode

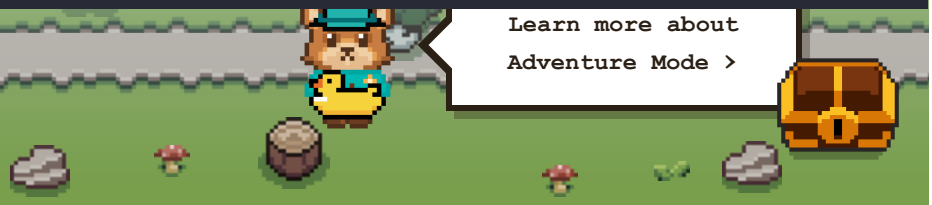
Coding fundamentals as bite-sized lessons and challenges.

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

League

Sign in to track progress



PLUS SPACES

GET CERTIFIED FOR TEACHERS

PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.